

Artificial Neural Networks

Basic Understanding, Working Mechanism & Real-World Examples

(ANN)

Presentation Agenda



Concept

Understanding the biological inspiration and basic architecture of neural networks.



Mechanism

Forward propagation, activation functions, loss calculation, and backpropagation.



Application

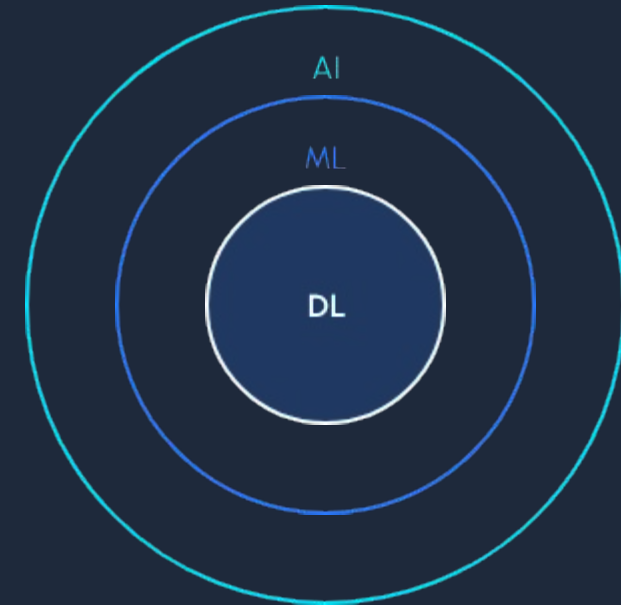
Training process, optimization, and a real-world handwriting recognition example.

Setting the Context

AI vs. ML vs. DL

To understand ANNs, we must place them in the hierarchy of Artificial Intelligence.

- ▶ **Artificial Intelligence (AI):** Any technique that enables computers to mimic human intelligence.
- ▶ **Machine Learning (ML):** A subset of AI that uses statistical methods to enable machines to improve with experience.
- ▶ **Deep Learning (DL):** A subset of ML that uses multi-layered neural networks (ANNs) to solve complex problems.



What is an ANN?

An **Artificial Neural Network (ANN)** is a computational model inspired by the way biological neural networks in the human brain process information.

Key Characteristics:

- ▶ Composed of interconnected processing nodes called **neurons**.
- ▶ Learns to perform tasks by analyzing training examples.
- ▶ Doesn't require task-specific rules to be programmed.



Information Processing System

Input → Hidden Processing → Output

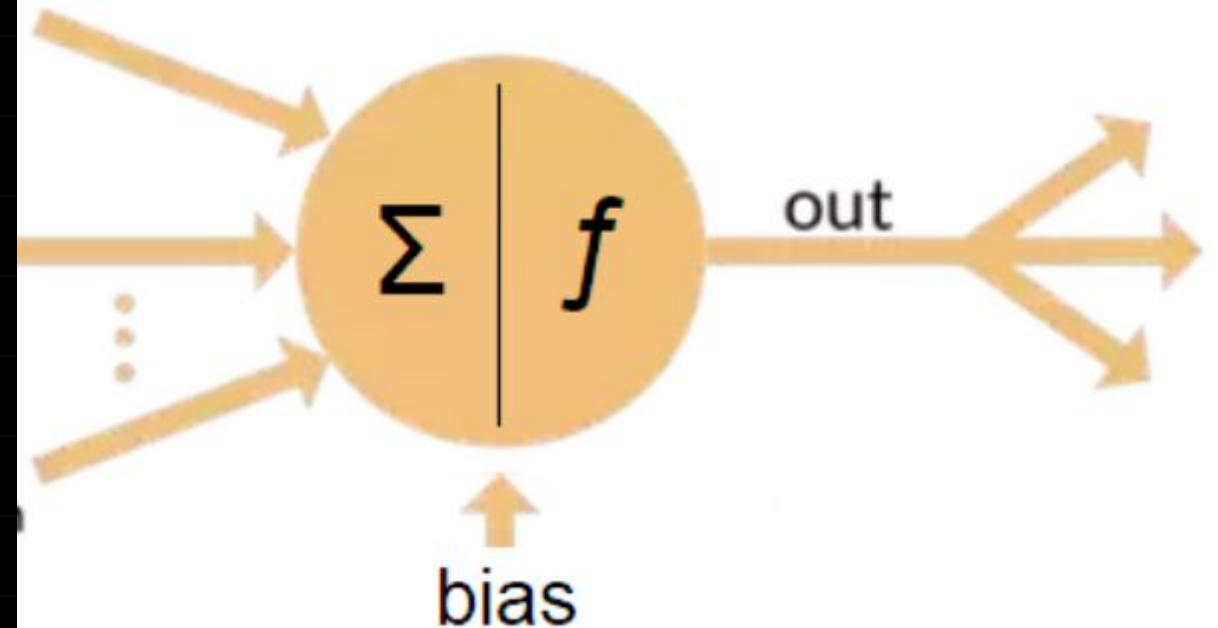
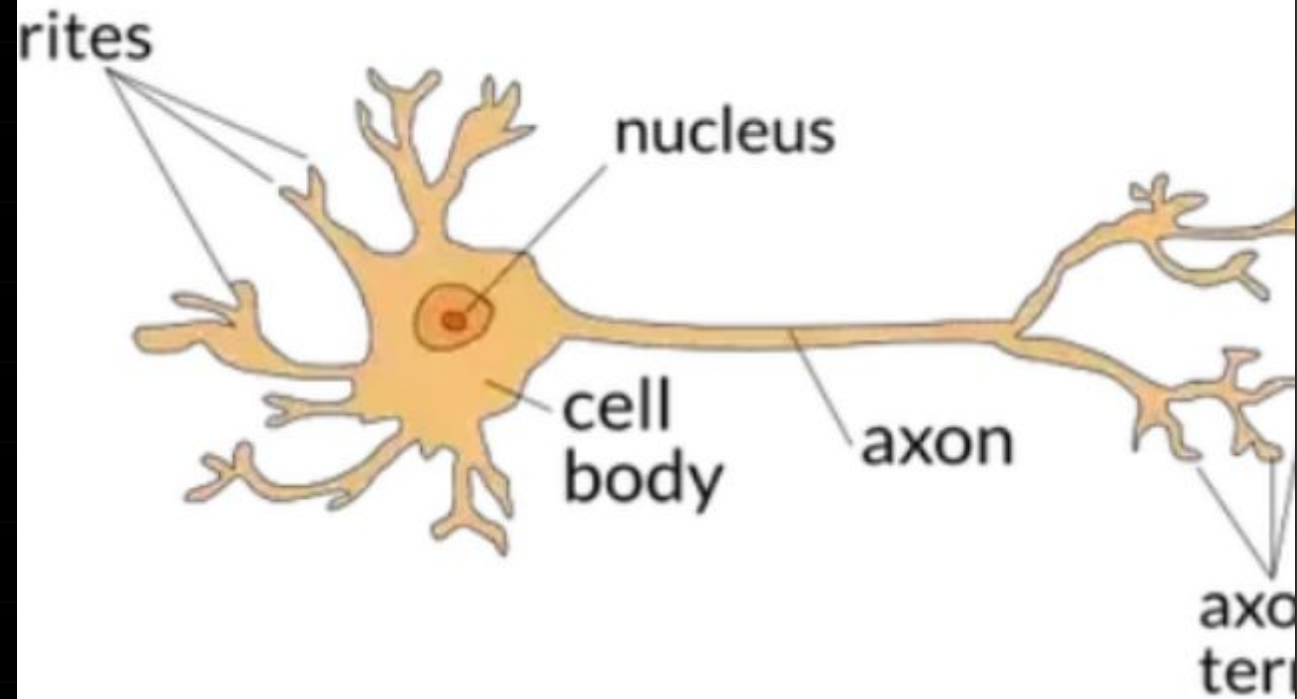
Biological Inspiration

The Human Neuron

Our brain consists of billions of neurons. Each neuron receives signals via **dendrites**, processes them in the **soma** (cell body), and if the signal is strong enough, fires an output signal through the **axon** to other neurons.

ANN Mapping:

- ▷ Dendrites $\approx$ Input Layer
- ▷ Nucleus/Soma $\approx$ Hidden Node (Summation)
- ▷ Axon $\approx$ Output/Activation

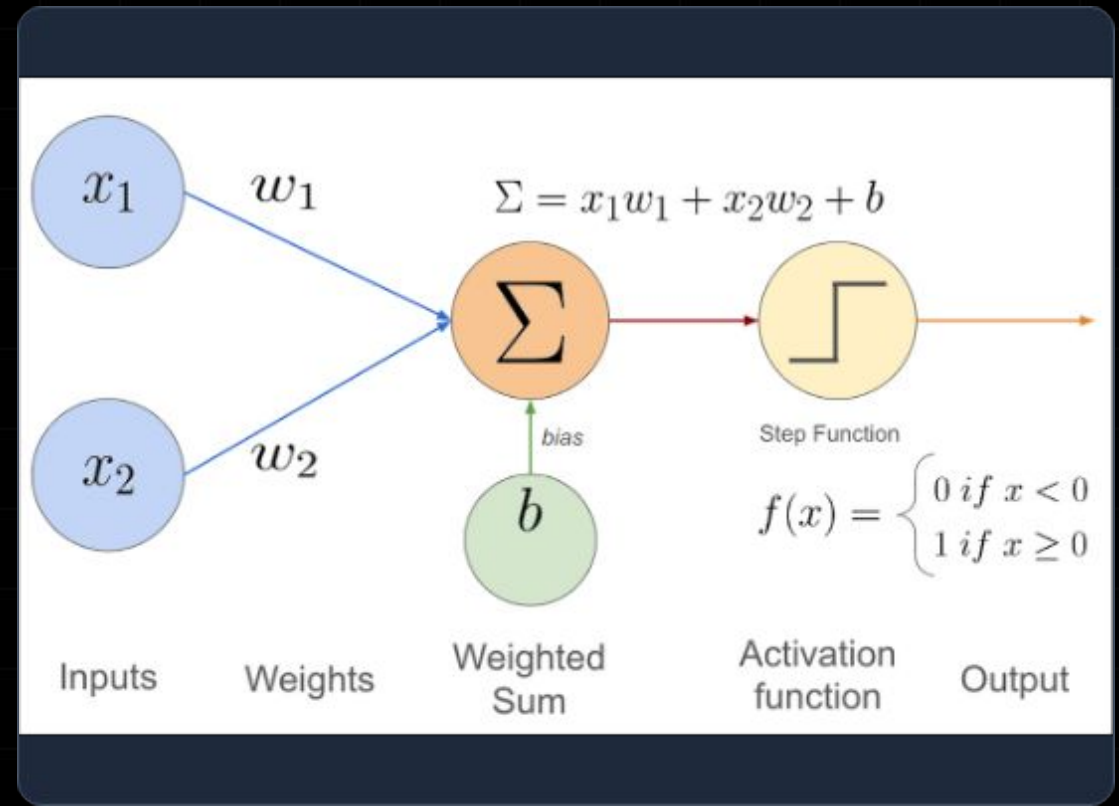


The Perceptron: The Building Block

The **Perceptron** is the simplest form of an ANN, consisting of a single neuron.

It was developed in the 1950s by Frank Rosenblatt. While modern networks are much deeper, understanding the perceptron is key to understanding the whole system.

It takes inputs, applies weights, sums them up, adds a bias, and passes the result through an activation function.



The Mathematical Model

The output of a single neuron can be expressed mathematically as:

$$y = f \left(\sum_{i=1}^n w_i x_i + b \right)$$

x_i

Inputs: The features or data points fed into the neuron.

w_i

Weights: Determine the importance of each input.

b

Bias: Allows the activation function to be shifted.

Component 1: Inputs & Weights

Inputs (Features)

Raw data fed into the network. For an image, this could be pixel intensity values. For a house, it could be square footage.

Weights (Strength)

Weights represent the **strength of the connection** between neurons. If a weight is high (absolute value), the input has a significant influence on the output.

"Learning" in a neural network essentially means adjusting these weights to get the correct output.

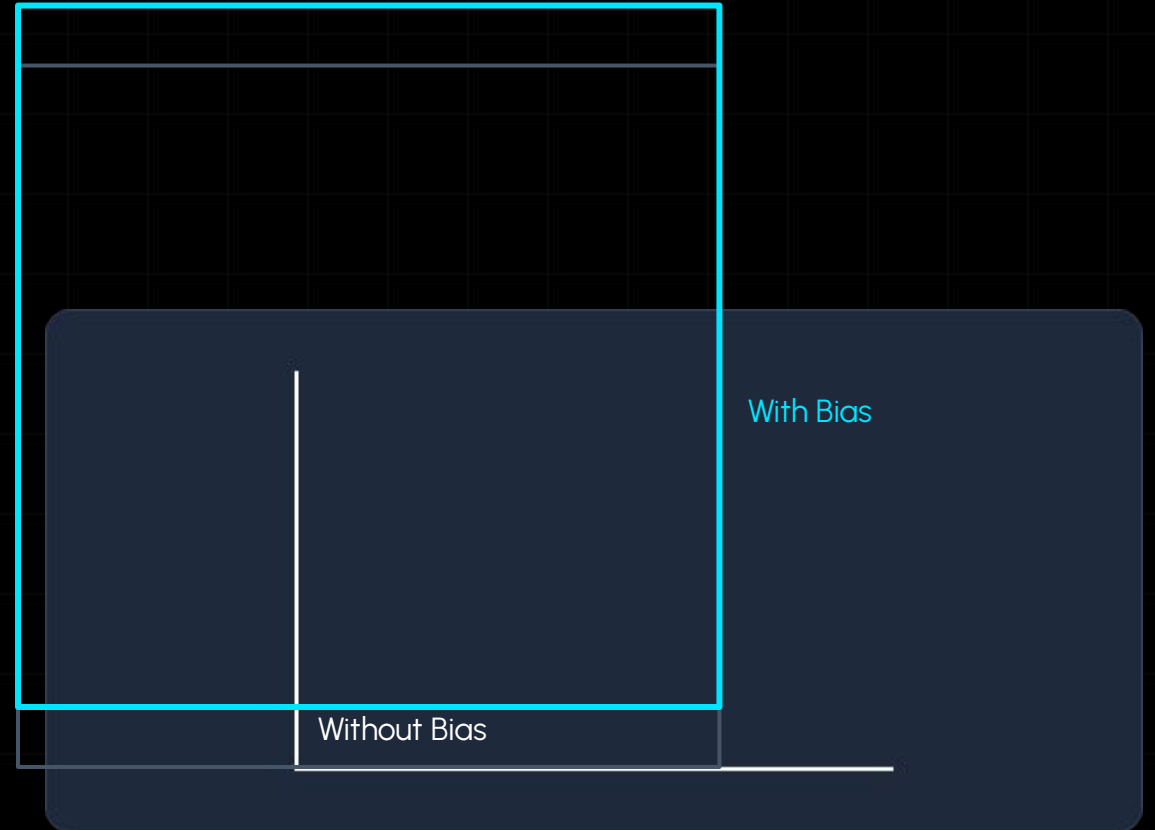
Component 2: Bias

Why do we need Bias?

Bias is similar to the **intercept** in a linear equation ().

$$y = mx + c$$

It allows the model to shift the activation function to the left or right, ensuring that the neuron can fire even when all inputs are zero, fitting the data better.



Component 3: Activation Function



The Decision Maker

The activation function decides whether a neuron should be activated or not. It calculates a "weighted sum" and adds bias and then decides whether it should fire.

Introducing Non-Linearity

Without activation functions, a neural network is just a giant **linear regression** model.

Real-world data (images, audio, text) is non-linear. Activation functions allow the network to learn complex patterns and curves.

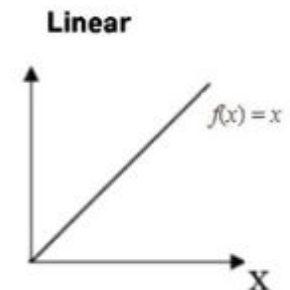
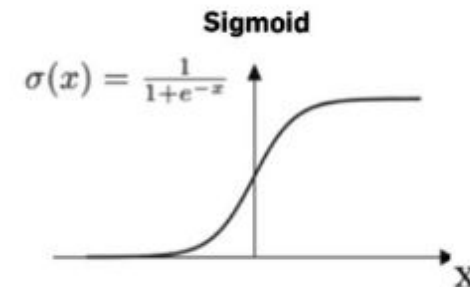
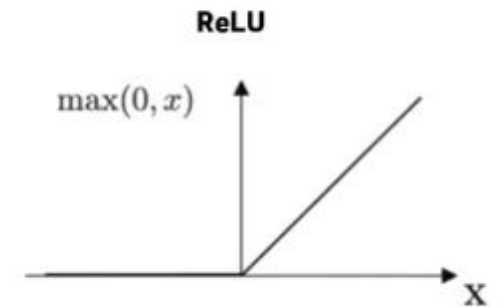
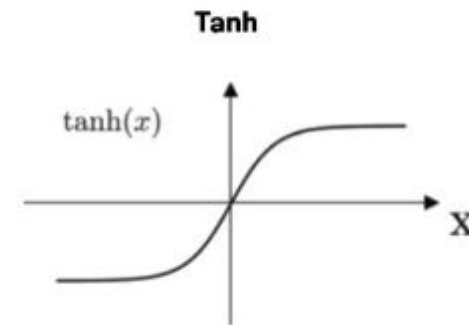
Common Functions: Sigmoid & Tanh

Sigmoid

Squashes values between **0 and 1**. Good for probability output.

Tanh (Hyperbolic Tangent)

Squashes values between **-1 and 1**. Zero-centered, often works better than Sigmoid in hidden layers.



Common Functions: ReLU

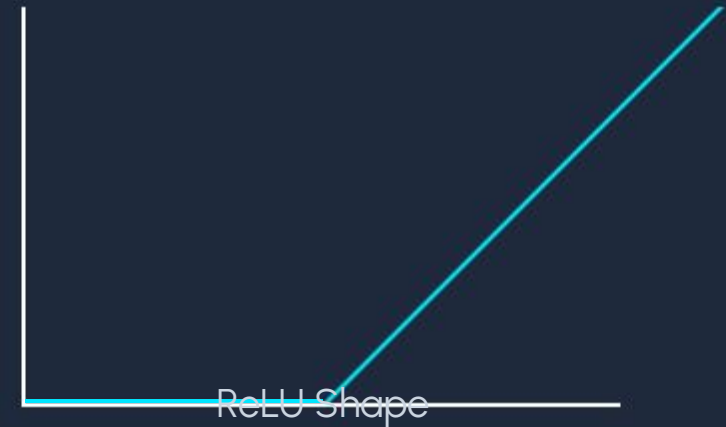
Rectified Linear Unit (ReLU)

Currently the most popular activation function for hidden layers.

$$f(x) = \max(0, x)$$

Why ReLU?

- ▶ Computationally efficient (simple math).
- ▶ Solves the "Vanishing Gradient" problem (we'll touch on this later).
- ▶ Only activates neurons if the input is positive.



Common Functions: Softmax

Softmax Function

Used primarily in the **Output Layer** for multi-class classification problems.

It converts a vector of raw numbers (logits) into a probability distribution.

Example: $[2.0, 1.0, 0.1] \rightarrow [0.7, 0.2, 0.1]$

The sum of all outputs equals 1.0 (100%).



Use Case

Classifying an image as a Cat, Dog, or Bird. The Softmax function gives the probability for each class.

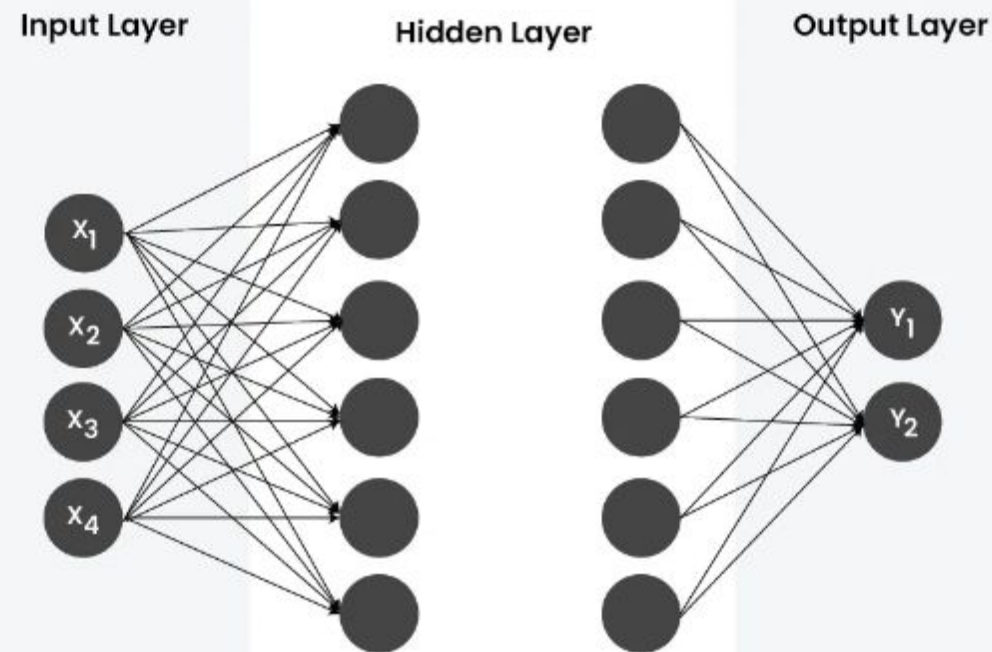
ANN Architecture

The Layers

- ▶ **Input Layer:** Receives raw data. No computation happens here.
- ▶ **Hidden Layer(s):** Where the magic happens. Neurons process features, extract patterns, and apply non-linearity.
- ▶ **Output Layer:** Produces the final prediction or classification.

A "Deep" Neural Network simply means it has many hidden layers.

Input Layer in an ANN



Deep Learning: Going Deeper

Shallow Network

1 Hidden Layer. Can solve simple problems but struggles with complex patterns like faces or speech.

Deep Network

Multiple Hidden Layers. Each layer learns increasingly complex features (Edges → Shapes → Objects).

The Trade-off

Deeper networks are more powerful but require more data and computing power to train.

How do ANNs Learn?

The learning process is a cycle of two main phases:

1. Forward Propagation



Make a guess. Data flows from input to output.

2. Backpropagation



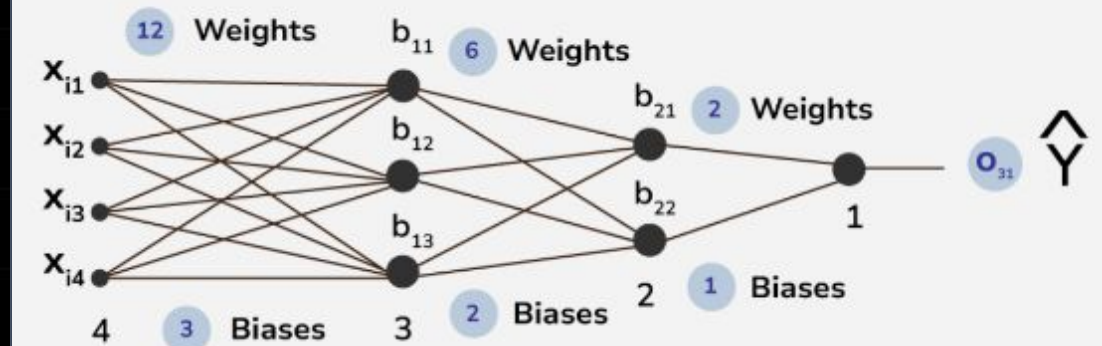
Learn from mistakes. Error information flows from output to input to update weights.

Forward Propagation

During forward propagation, input data enters the network.

At each layer, the data is multiplied by weights, bias is added, and the result is passed through an activation function.

This continues layer by layer until the output layer generates a prediction.



Imagine a simple neuron with 2 inputs:

Inputs

$$x_1 = 0.5$$

$$x_2 = 0.2$$

Weights & Bias

$$w_1 = 0.8$$

$$w_2 = 0.4$$

$$b = 0.1$$

$$\text{Step 1: Sum} = (0.5 * 0.8) + (0.2 * 0.4) + 0.1$$

$$\text{Sum} = 0.4 + 0.08 + 0.1 = \mathbf{0.58}$$

Step 2: Activation (Sigmoid of 0.58)

The Loss Function (Cost)

How wrong are we?

The network made a guess (Prediction). We have the correct answer (Target/Label).

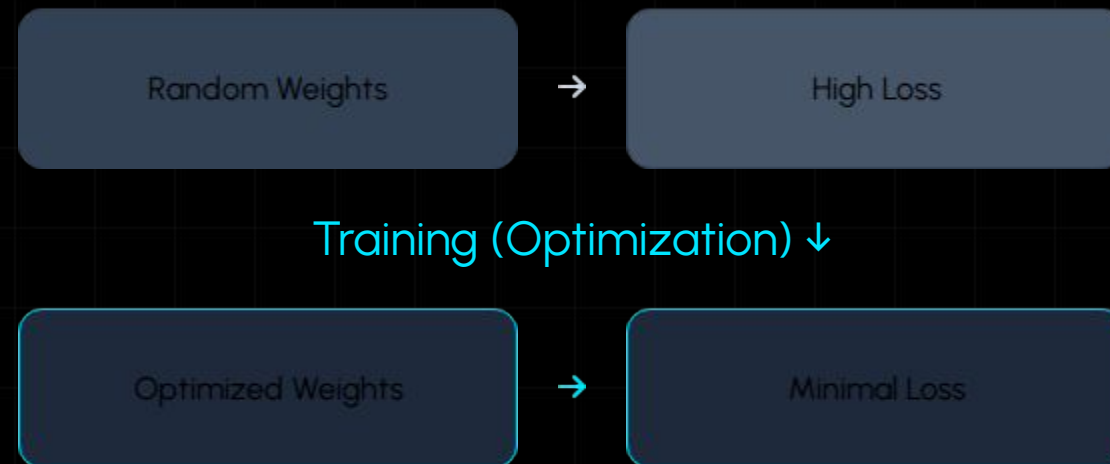
The **Loss Function** calculates the difference between the prediction and the target.

Common Loss Functions

- ▶ **Mean Squared Error (MSE):** For Regression (predicting prices). $(y - \hat{y})^2$
- ▶ **Cross-Entropy:** For Classification (cat vs dog).

The Goal: Minimize Loss

We want the lowest possible error.



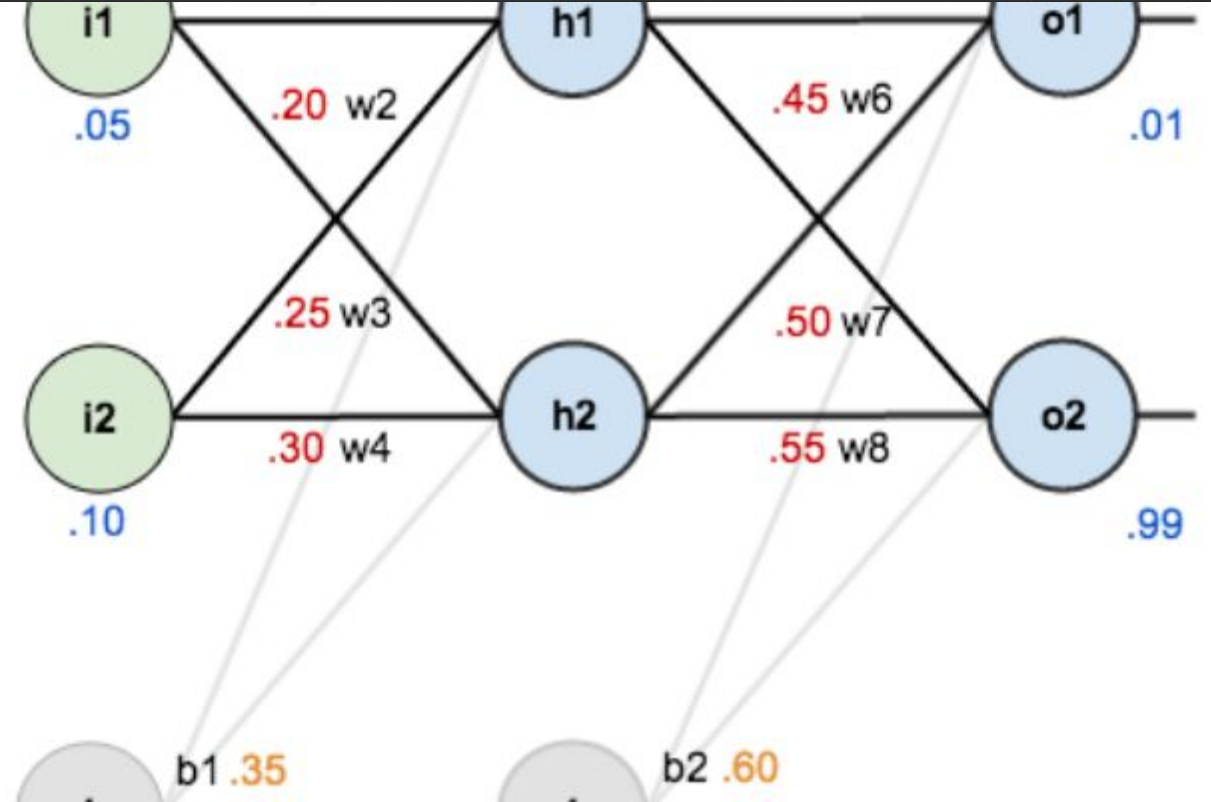
Backpropagation

The Heart of Learning

Once we know the loss, we need to tell the network **how to adjust the weights** to reduce that loss.

Backpropagation calculates the gradient (slope) of the loss function with respect to each weight.

It moves backward from Output → Hidden → Input, distributing the "blame" for the error.



The Chain Rule

Backpropagation relies on the Calculus concept called the Chain Rule.

$$\frac{\partial \text{Loss}}{\partial \text{Weight}} = \frac{\partial \text{Loss}}{\partial \text{Output}} \times \frac{\partial \text{Output}}{\partial \text{Hidden}} \times \frac{\partial \text{Hidden}}{\partial \text{Weight}}$$

"How much does the Loss change if the Weight changes?"

It's like a domino effect calculation.

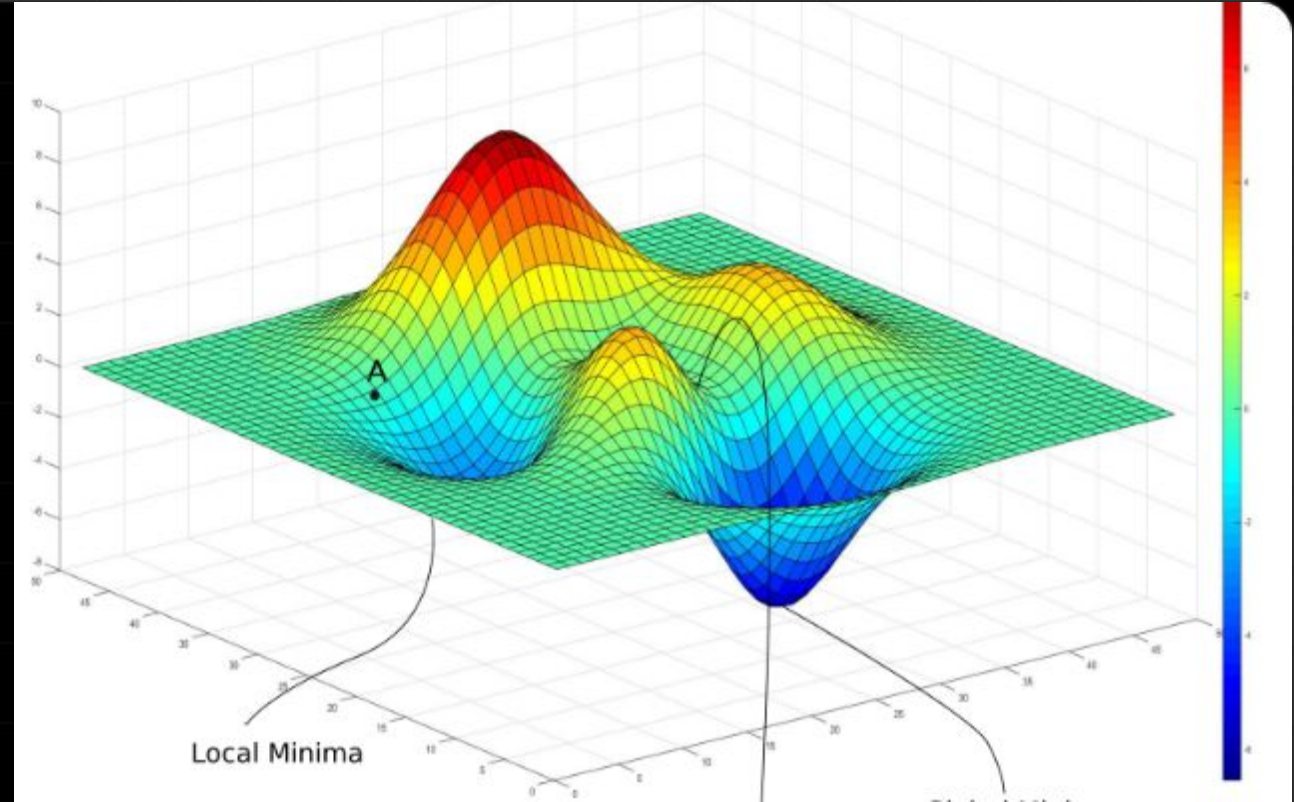
The Optimizer: Gradient Descent

Descending the Mountain

Imagine you are on top of a mountain (High Loss) in the fog.
You want to reach the valley (Low Loss).

Gradient Descent is the algorithm that tells you which direction to step to go downhill.

- ▶ Calculate slope (Gradient).
- ▶ Take a step in the opposite direction.
- ▶ Repeat.



Learning Rate (Alpha)

Step Size

The Learning Rate (α) determines how big of a step we take during Gradient Descent.

$$w_{\text{new}} = w_{\text{old}} - \alpha \times \text{gradient}$$

Too High

We might overshoot the minimum and never converge (bounce around).

Too Low

Training will take forever to reach the bottom.

The Training Loop



Epoch

One complete pass of the ENTIRE training dataset through the neural network.



Batch Size

We can't feed all data at once. We split it into batches (e.g., 32 images at a time).



Iteration

Number of batches needed to complete one epoch.

Generalization

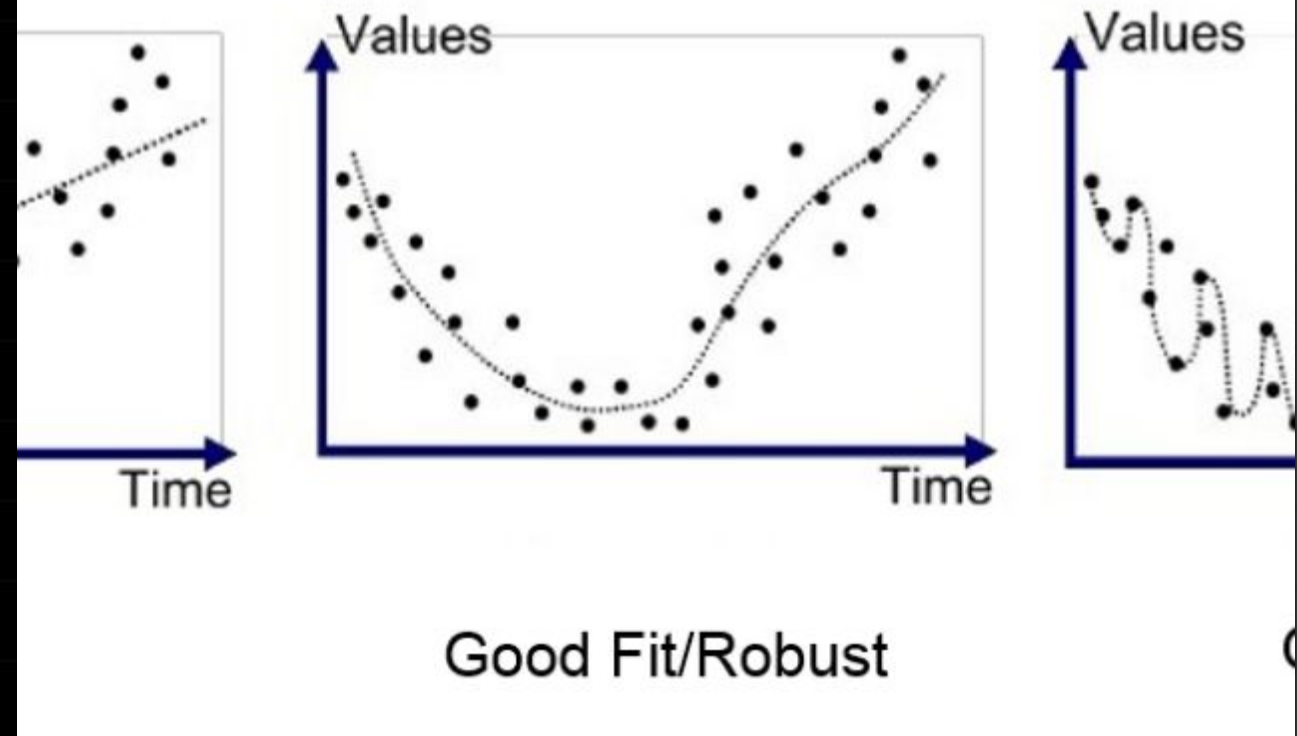
The goal is not to memorize the data, but to learn the pattern.

Underfitting

Model is too simple. Can't capture the trend (High Bias).

Overfitting

Model is too complex. Memorizes noise and outliers (High Variance). Fails on new data.



Real World Example: MNIST

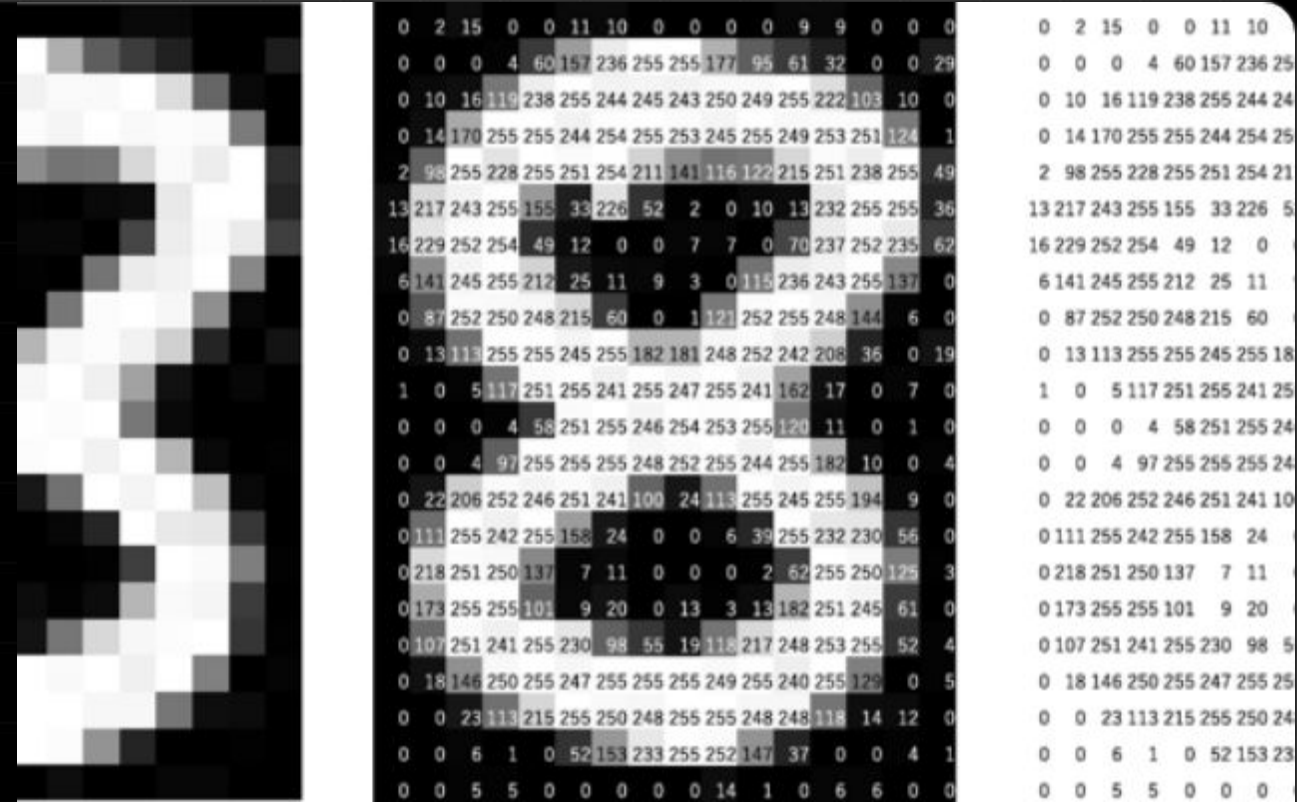
"Hello World" of Deep Learning

The MNIST dataset consists of 70,000 handwritten digits (0-9).

Goal: Train an ANN to recognize the digit from a 28x28 pixel image.

Input: 784 neurons (28x28 pixels flattened).

Output: 10 neurons (Probabilities for 0-9).

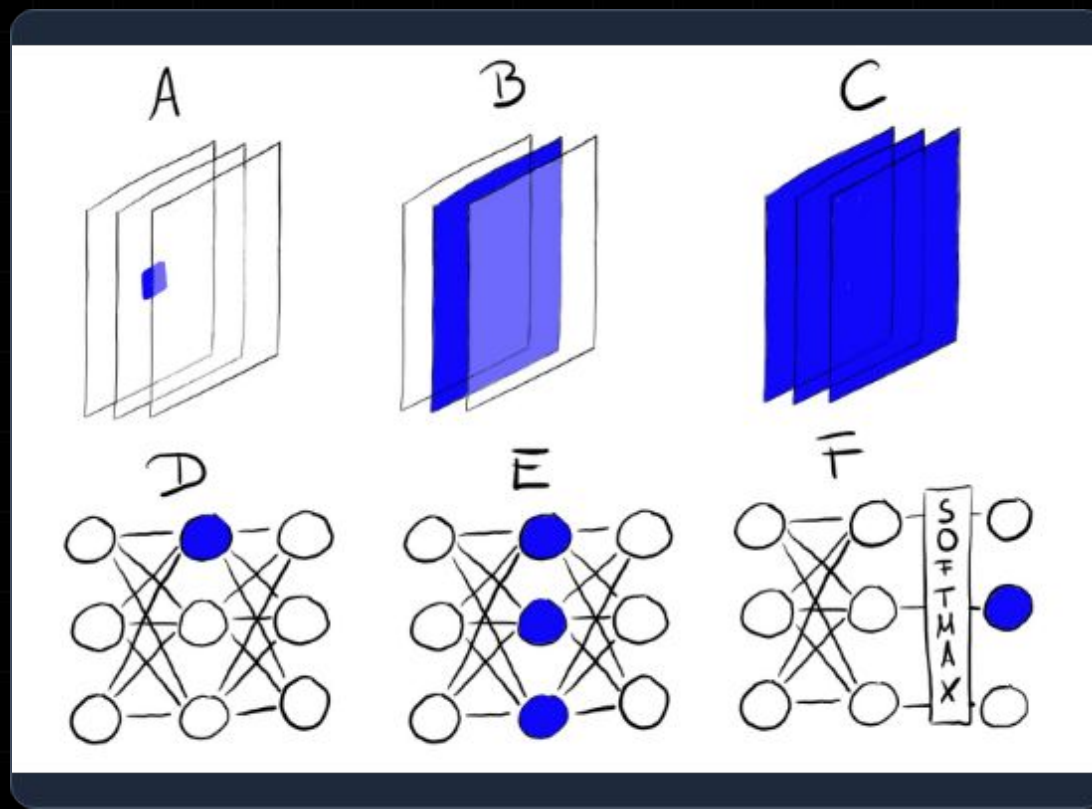


What the Network Sees

Hierarchical Learning

As we move deeper into the network, the features become more abstract.

- ▶ **Layer 1:** Detects edges, curves, and lines.
- ▶ **Layer 2:** Detects loops and shapes (e.g., the top of an '8').
- ▶ **Layer 3:** Assembles shapes into complete digits.



Applications of ANN



Healthcare

Tumor detection in X-rays, drug discovery, genetic analysis.



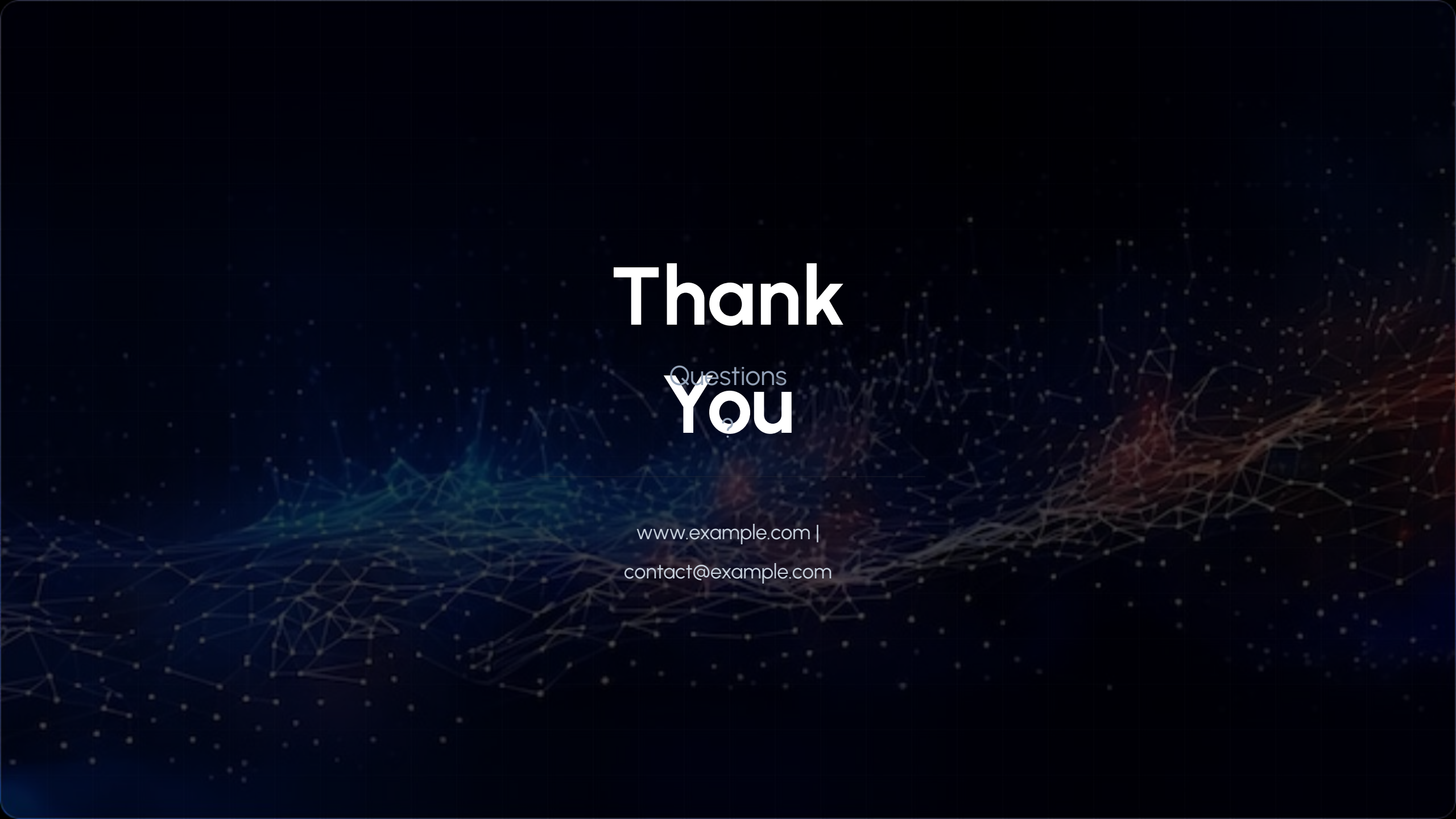
Finance

Stock price prediction, fraud detection, credit scoring.



Automotive

Self-driving cars (Object detection, path planning).



Thank

Questions
You

www.example.com |
contact@example.com